Assignment - 3

Name : Prachiti Suryawanshi
PRN : 12622521 77
Class : S.Y. B.Tech CSE-CSF

Problem Statement : Write a C program to create a student database using an array of structures. Apply searching (Linear & Binary Search) and sorting techniques (Insertion Sort, Selection Sort, Shell sort).

Objective :
1) To understand use of an array of structures for maintaining records.
2) To implement & analyze searching techniques.
3) To implement & analyze sorting techniques.
4) To Compare Insertion Sort, Selection Sort, Shell Sort.

Theory : Searching & sorting are two of the most fundamental operations in data structures and algo. They are widely used in computer applications such as database management systems, search engines & information retrieval systems. Efficient searching & sorting techniques improves the performance of software by reducing the time required to locate & organize data.

Searching : Searching is the process of finding the location of a specific element called the key.

FOR EDUCATIONAL USE

If the element is present, the search operation returns its position; otherwise, it indicates that the element is not found.

Searching is an essential operation in applications such as student record management, library systems, banking systems, inventory management & online shopping.

Types of Searching :

1) Linear Search - Linear search, also known as Searching Search, is the simplest technique. In this method, each element of the list is compared one by one with the target element until a match is found or the entire list has been searched.

2) Binary Search - Binary search is an efficient searching algorithm that works only on sorted data. It repeatedly divides the search interval into two halves until the target element is found or the search space becomes empty.

Sorting : Sorting is the process of arranging data elements in a specific order, usually ascending or descending. Sorted data allows faster searching, easier analysis & efficient data processing.

Sundaram
FOR EDUCATIONAL USE

Types of Sorting :

1) Bubble Sort - It repeatedly compares adjacent elements & swaps them if they are in the wrong order. After each pass, the largest element moves to its correct position, similar to bubbles rising to the surface.

2) Selection Sort - It repeatedly selects the smallest element from the unsorted portion of the list & places it at the beginning.

3) Insertion Sort - It builds a sorted list one element at a time by inserting each new element into its correct position in the already sorted portion of the array.

4) Shell Sort - Shell Sort allows the exchange of elements that are far apart. It sorts elements by comparing & swapping items seperated by a specific gap, graduallly reducing the gap until it becomes 1.

5) Merge Sort - It is a divide & conquer algorithm that divides the array into smaller subarrays, sorts each subarray recursively & then merges them into a sorted array.

FOR EDUCATIONAL USE

6) Quick Sort - It is another divide & conquer algorithm. It selects a pivot element and partitions the array into two parts such that elements smaller than the pivot are placed before it & larger elements after it.

Platform : • 64-bit Open source Linux or its derivative
• Open Source C Programming tool like gcc Editor

Pseudo Code :

```
Step 1 - Start
Step 2 - Accept values from user
Step 3 - Linear_sort (array, target)
        for (i=0 to len(array) - 1)
            if array[i] == target
                return ;
            end
        end.
        return -1
    end
Step 4 - bubble sort (array)
    n = len(array)
    for (i = 0 to n-1)
        swapped = False
        for j=0 to n-i-2
            if array (j) > array (j+1)
                temp = array[j]
```

Sundaram
FOR EDUCATIONAL USE

```
            array[j] = array[j+1]
            array[j+1] = temp
            swapped = true
        end
    end
    if swapped == false
        break;
    end
  end
end
```




```
ep 5 - insertion_sort(array)
  n = length(A)
  for (i = 1 to n-1)
      key = A[i]
      j = i-1
      while j >= 0 & A[j] = key
          A[j+1] = A[j]
          j = j-1
      end
      A[j+1] = key
  end
tep 6 - selection_sort(array)
  n = length(array)
  for i=0 to n-2
      for j = 1 to n-1
          if array[j] < array[min_index]
              min_index = j
          end
      end
  end
```

Sundaram
FOR EDUCATIONAL USE

```
        if min_index != i
            swap array[i] with a[min_index]
        end
    end
```

Step 7 -

```
shell_sort (array, n)
{
  gap = floor (n/2)
  while gap > 0
     for i = gap to n-1
         temp = array[i]
         j = i
         while j >= gap & array[j-gap] > te
             array[j] = array[j-gap]
             j = j-gap
         end
         array[j] = temp
     end
     gap = floor (gap/2)
end
```

Conclusion : Thus, implemented different searchi & sorting methods on the student database. This System is able to perform sorting under different cases.

FOR EDUCATIONAL USE

FAQs:

1. What is the meaning of database? How to maintain in C?

Database is an organised collection of related data stored for easy access, updating & management.
eg. Student records containing roll no., name, marks & department.

How to maintain it:
- Use a structure
- Use an array of func. to store records.
- Perform operations
- Store data temporarily in memory or permanantly using files.

What are the applications of searching?

- Student database management
- Library management
- Banking & ATM systems
- Employee record system
- Contact list system

EDUCATIONAL USE

3) Compare between linear & binary search.

Features	Linear	Binary
Data requirement	Sorted	Sorted
Method	One by one	Divides space in
Time complexity	O(n)	O(logn)
Speed	Slower	Faster
Implementation	Simple	Moderate
Suitable for	Small list	Large list
Memory	No extra	No extra

4) What are the applications of sorting?

- Fast searching
- Student marks arrangement
- Database indexing
- Payroll processing
- Data analysis

5) Compare selection sort, insertion sort, shell sor

Features	Selection	Insertion	Shell
Bad case	$O(n^2)$	$O(n)$	$O(n \log n)$
Average case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst case	$O(n^2)$	$O(n^2)$	$O(n^2)$

Sundaram
FOR EDUCATIONAL USE

Stable	No	Yes	No
Inplace	Yes	Yes	Yes
Efficiency	Poor	Good	Better

6) What is external & internal sorting, in place, stable features of sorting algorithm?

Internal Sorting :
Entire data fits in main memory
Eg. Insertion

External Sorting :
Data is too large to fit in ram then secondary storage.
Eg. External merge sort.

Stable Sorting :
Preserves relative order of equal elements.
Eg. Insertion sort

Unstable Sorting :
May change the order of equal elements.
Eg. Selection sort

30/7/24

FOR EDUCATIONAL USE

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

void display(struct Student s[], int n) {
    int i;
    printf("\nRoll No.\tName\t\tMarks\n");
    for (i = 0; i < n; i++)
        printf("%d\t\t%s\t\t%.2f\n",
               s[i].roll, s[i].name, s[i].marks);
}

void linearSearch(struct Student s[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++) {
        if (s[i].roll == key) {
            printf("Student found: %s, Marks: %.2f\n",
                   s[i].name, s[i].marks);
            return;
        }
    }
    printf("Student not found.\n");
}

void binarySearch(struct Student s[], int n, int
key) {
    int low = 0, high = n - 1, mid;
```

```
    while (low <= high) {
        mid = (low + high) / 2;

        if (s[mid].roll == key) {
            printf("Student found: %s, Marks: %.2f\n",
                   s[mid].name, s[mid].marks);
            return;
        } else if (s[mid].roll < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    printf("Student not found.\n");
}

void insertionSort(struct Student s[], int n) {
    int i, j;
    struct Student key;

    for (i = 1; i < n; i++) {
        key = s[i];
        j = i - 1;

        while (j >= 0 && s[j].roll > key.roll) {
            s[j + 1] = s[j];
            j--;
        }
        s[j + 1] = key;
    }
}
```

```
void selectionSort(struct Student s[], int n) {
    int i, j, min;
    struct Student temp;

    for (i = 0; i < n - 1; i++) {
        min = i;

        for (j = i + 1; j < n; j++) {
            if (s[j].roll < s[min].roll)
                min = j;
        }

        temp = s[i];
        s[i] = s[min];
        s[min] = temp;
    }
}

void shellSort(struct Student s[], int n) {
    int gap, i, j;
    struct Student temp;

    for (gap = n / 2; gap > 0; gap /= 2) {
        for (i = gap; i < n; i++) {
            temp = s[i];
            j = i;

            while (j >= gap &&
                   s[j - gap].roll > temp.roll) {
                s[j] = s[j - gap];
                j -= gap;
            }
            s[j] = temp;
```

```c
int main() {
    struct Student s[100];
    int n, i, choice, key;

    printf("Enter number of students (1-100): ");
    scanf("%d", &n);

    if (n < 1 || n > 100) {
        printf("Invalid number of students.\n");
        return 0;
    }

    for (i = 0; i < n; i++) {
        printf("\nEnter roll number: ");
        scanf("%d", &s[i].roll);

        printf("Enter name (without spaces): ");
        scanf("%49s", s[i].name);

        printf("Enter marks: ");
        scanf("%f", &s[i].marks);
    }

    do {
        printf("\n--- STUDENT DATABASE ---\n");
        printf("1. Display records\n");
        printf("2. Linear Search\n");
        printf("3. Insertion Sort\n");
        printf("4. Selection Sort\n");
        printf("5. Shell Sort\n");
        printf("6. Binary Search\n");
        printf("0. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
```

```
    switch (choice) {
        case 1:
            display(s, n);
            break;

        case 2:
            printf("Enter roll number to search: ");
            scanf("%d", &key);
            linearSearch(s, n, key);
            break;

        case 3:
            insertionSort(s, n);
            printf("Sorted using Insertion Sort.\n");
            display(s, n);
            break;

        case 4:
            selectionSort(s, n);
            printf("Sorted using Selection Sort.\n");
            display(s, n);
            break;

        case 5:
            shellSort(s, n);
            printf("Sorted using Shell Sort.\n");
            display(s, n);
            break;
```

```
        case 6:
            printf("Sort records first using option 3, 4
or 5.\n");
            printf("Enter roll number to search: ");
            scanf("%d", &key);
            binarySearch(s, n, key);
            break;

        case 0:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice.\n");
    }
  } while (choice != 0);

  return 0;
}
```